

ROOTで解析してみよう

サマーチャレンジ 2026 演習#10
ミューオンの寿命測定と歳差運動
九州大学 素粒子実験研究室
文責：久戸瀬太一、五島征一郎

ROOTとは？

- ・ 欧州原子核研究機構（**CERN**）が開発した解析ソフトウェア・ライブラリ群
- ・ 独自のデータ構造を持ち、**多イベント・多変数**の処理に強い
- ・ 素粒子・原子核・宇宙（素核宇）など、高エネルギー分野での標準的な解析ツール

以下の環境で動作

- ・ Linux (WSL2でもOK)
 - ・ macOS 14 Sonoma以降
 - ・ Windows
- こだわりがなければWSL2を推奨

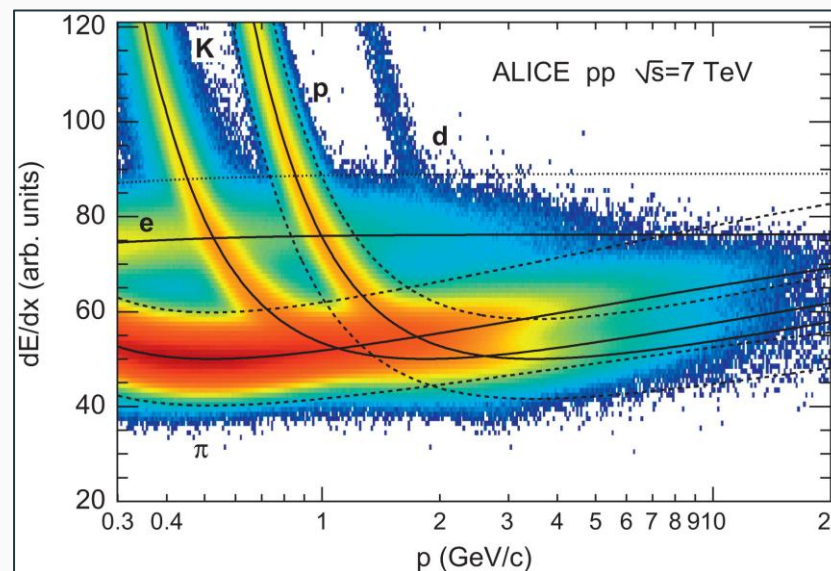
ここでは2通りの導入方法を紹介：

- ①パッケージマネージャを利用する
- ②ビルド済みバイナリをダウンロードする

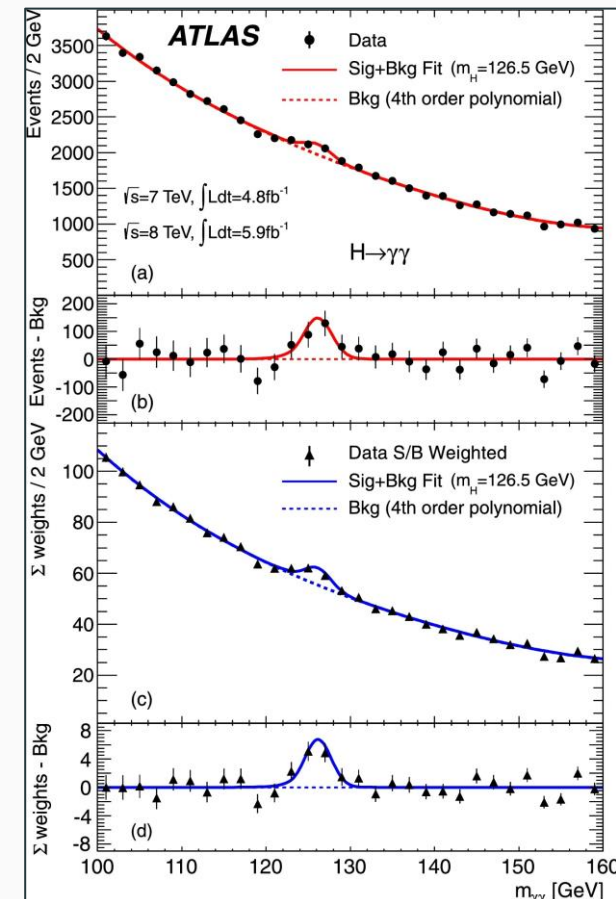
*オープンソースソフトウェアなので、上級者はgit cloneしてもよい



ROOTを使った解析例↓→



pp衝突で生じた荷電粒子を運動量、エネルギー損失についてプロットしたもの。実線は理論曲線、ヒートマップは各(p, dE/dx)で観測された荷電粒子数を表す。粒子ごとに異なる帯構造を作り、識別が可能である。



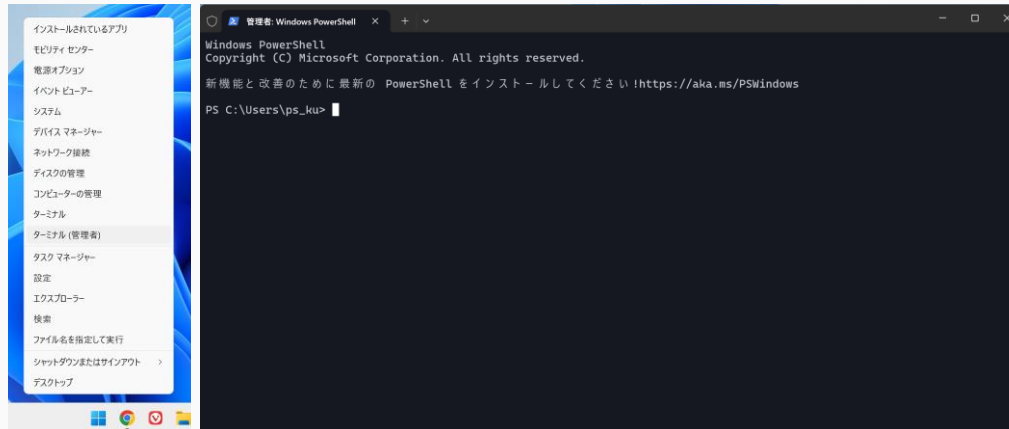
ATLAS実験による $H \rightarrow \gamma\gamma$ 候補事象の不変質量分布 [2]。126.5 GeV 付近に背景から有意な超過が見られる。Higgs粒子発見の決定打。

【準備】WSL2の導入（Windowsユーザ向け）

Windows Subsystem for Linux 2

- ・ ・ ・ Windows上でLinux環境を動かす、仮想環境

① ターミナル（管理者）を起動する（を右クリック）



② `wsl --install --distribution AlmaLinux-10`を実行。

完了後、再起動する。

③ ユーザ名とパスワードを求められるので、設定する。

パスワードはこの後使うので忘れないこと。

④ 「`Ctrl` + `,`>プロファイル>(使用しているLinuxOS)>開始ディレクトリ」の「親プロセス ディレクトリの使用」からチェックを外し、`~`に設定・保存する。またターミナルで`cd ~`を実行する。

Tips：ディストリビューションの指定

`--distribution`以降を書かなければ、Ubuntuがインストールされる。世界シェアではUbuntuが多数を占めるが、自然科学分野ではCentOS（サポート終了）やAlmaLinuxが主流である。Ubuntuユーザ向けの手順も併記するので、どちらを選んでもよい。

Tips：④の解説

Linuxから見れば、Cドライブは外部のファイルシステムである。管理方法の違いから読み書きには時間がかかるので、作業はLinuxのホームディレクトリ「`~`」で行うことが推奨される。
`cd`はchange directoryの略で、`cd ~`は「`~`に移動せよ」と読める。

【発展】 Visual Studio Codeを使ってみよう

VSCoideを使えばコードの編集とターミナルの操作が1つのウィンドウで完結する。

Linux / macOS / Windowsユーザー

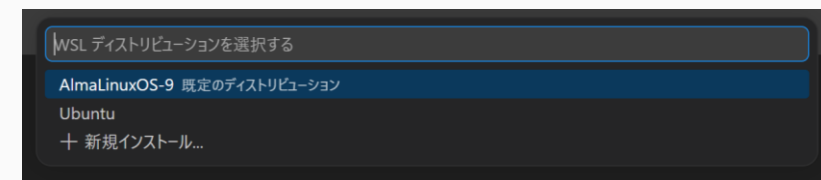
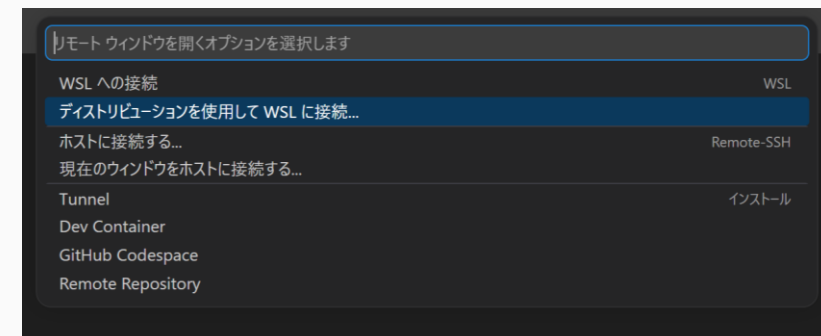
- ① VSCoideを起動する。
- ② 「ターミナル>新しいターミナル」をクリック。
ウィンドウ下部でターミナルが起動する。
- ③ `cd ~`を実行し、ホームディレクトリに移動する。



ターミナルの開き方

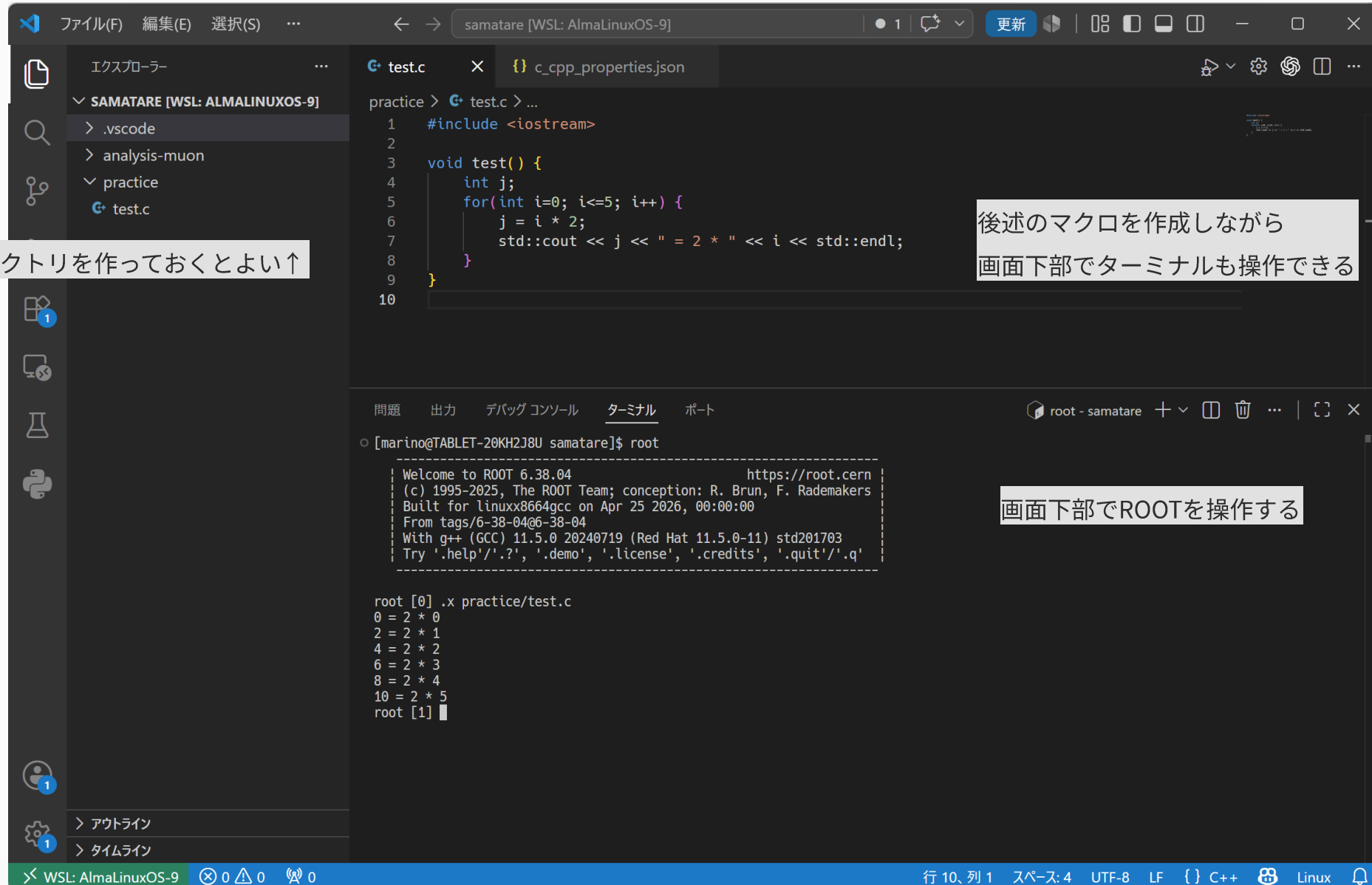
WSL2ユーザー

- ① VSCoideを起動する。
- ② ウィンドウ左下のをクリックし、
「ディストリビューションを使用してWSLに接続...>AlmaLinux-10」を選択
- ③ 「ターミナル>新しいターミナル」をクリック。
ウィンドウ下部でターミナルが起動する。
- ④ `cd ~`を実行し、ホームディレクトリに移動する。



WSL2の起動 in VSCode

VSCodeを使った開発例



パッケージマネージャ編：AlmaLinux / RHEL系, macOS, WSL2

ROOTコミュニティにより、パッケージが提供されている。

① 以下を実行する

\$は行頭記号なので入力しない

AlmaLinux / RHEL系

```
$ sudo dnf update
$ sudo dnf install epel-release
$ sudo dnf install root python3-root
```

macOS

```
$ brew install root
```

Is this ok [y/n]: はインストールの可否→yを入力する。

※エラーが出る！→FAQを参照。解決しなければビルド済みバイナリ編 for Alma...を参照

※Ubuntu / Debian系はパッケージが提供されていない。

→ビルド済みバイナリ編 for Ubuntuを参照のこと。

② rootを実行し、ROOTが起動することを確認する。

```
[marino@TABLET-20KH2J8U ~]$ root
-----
| Welcome to ROOT 6.38.04                               https://root.cern |
| (c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for linuxx86_64gcc on Apr 22 2026, 00:00:00 |
| From tags/6-38-04@6-38-04 |
| With g++ (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4) std201703 |
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'.' |
-----
root [0] █
```

※起動したけどなんか変

→FAQを参照。

.qで終了する。早くできた人は、demoで機能を試してみるとよい。

Tips：パッケージマネージャとは

プログラムやメタデータなどをひとまとめにしたデータ群をパッケージという。OSにはパッケージの一元管理システムであるパッケージマネージャが搭載されている。

Tips：①の解説

パッケージのインストールはコンピュータに変更を加えるので、管理者の承認が必要。sudoはsuperuser doの略で、管理者権限で実行することを意味する。

dnf / apt / Homebrewは各OSに搭載されたパッケージマネージャ。つまり左のコマンドは「管理者が命令する。dnfを使ってepel-releaseをinstallせよ。」と読める。

Tips：古いLinuxコンピュータを使っている方へ

古いRHEL系ではdnfの代わりにyumを使うことがある。研究室の古いコンピュータではCentOSやScientific Linuxが搭載されていることがあり、これらはyumを使う。AlmaLinuxでも利用可能だが、非推奨。

ビルド済みバイナリ編 for AlmaLinux / RHEL

- ① 前提パッケージを導入する。必要なパッケージは公式サイトに掲載されている：[Dependencies – ROOT](#)

AlmaLinux / RHEL系

```
$ sudo dnf update
$ sudo dnf install -y git make cmake gcc-c++ gcc binutils libX11-devel libXpm-devel
libXft-devel libXext-devel python openssl-devel xrootd-client-devel xrootd-libs-devel
```

- ② ホームディレクトリに opt/ を作り、移動。

[公式サイト](#)で自分の環境向けのファイル名を確認し、ダウンロード・展開する。

AlmaLinux / RHEL系

```
$ mkdir ~/opt
$ cd ~/opt
$ wget <公式サイトで確認したURL>
$ tar -xvf <ダウンロードした配布ファイル>
```

|→これより後ろは自分のOSに合ったものを公式サイトで確認する

- ③ PATHを通し、起動確認。

AlmaLinux / RHEL系

```
$ ROOTの環境設定を登録
$ source ~/シェル設定
$ root
```

.qで終了する。早くできた人は.demoで機能を試してみるとよい。

※起動したけどなんか変 / 起動しない→FAQを参照

Tips : Linuxコマンドを覚えよう

mkdir(make directory): ディレクトリ (フォルダ) を作る
wget(www get): ウェブからファイルを取得する
tar(tape archive): tar形式のファイル进行处理する。tar.gzはtar/gzip形式の圧縮ファイルで、xvfはextract (展開) , verbose (ログ表示) , file (ファイルを指定)

ビルド済みバイナリ編 for Ubuntu

- ① 前提パッケージを導入する。必要なパッケージは公式サイトに掲載されている：[Dependencies – ROOT](#)

Ubuntu / Debian系

```
$ sudo apt update
$ sudo apt install -y binutils cmake dpkg-dev g++ gcc libssl-dev git libx11-dev
libxext-dev libxft-dev libxpm-dev python3 libtbb-dev libvdt-dev libgif-dev
```

- ② ホームディレクトリに opt/ を作り、移動。Ubuntuのバージョンは `lsb_release -a` で確認できる。
[公式サイト](#) で自分の環境向けのファイル名を確認し、ダウンロード・展開する。

Ubuntu / Debian系

```
$ mkdir ~/opt
$ cd ~/opt
$ wget <公式サイトで確認したURL>
$ tar -xvf <ダウンロードした配布ファイル>
```

|→これより後ろは自分のOSに合ったものを公式サイトで確認する

- ③ PATHを通し、起動確認。

Ubuntu / Debian系

```
$ ROOTの環境設定を登録
$ source ~/シェル設定
$ root
```

.qで終了する。早くできた人は、demoで機能を試してみるとよい。

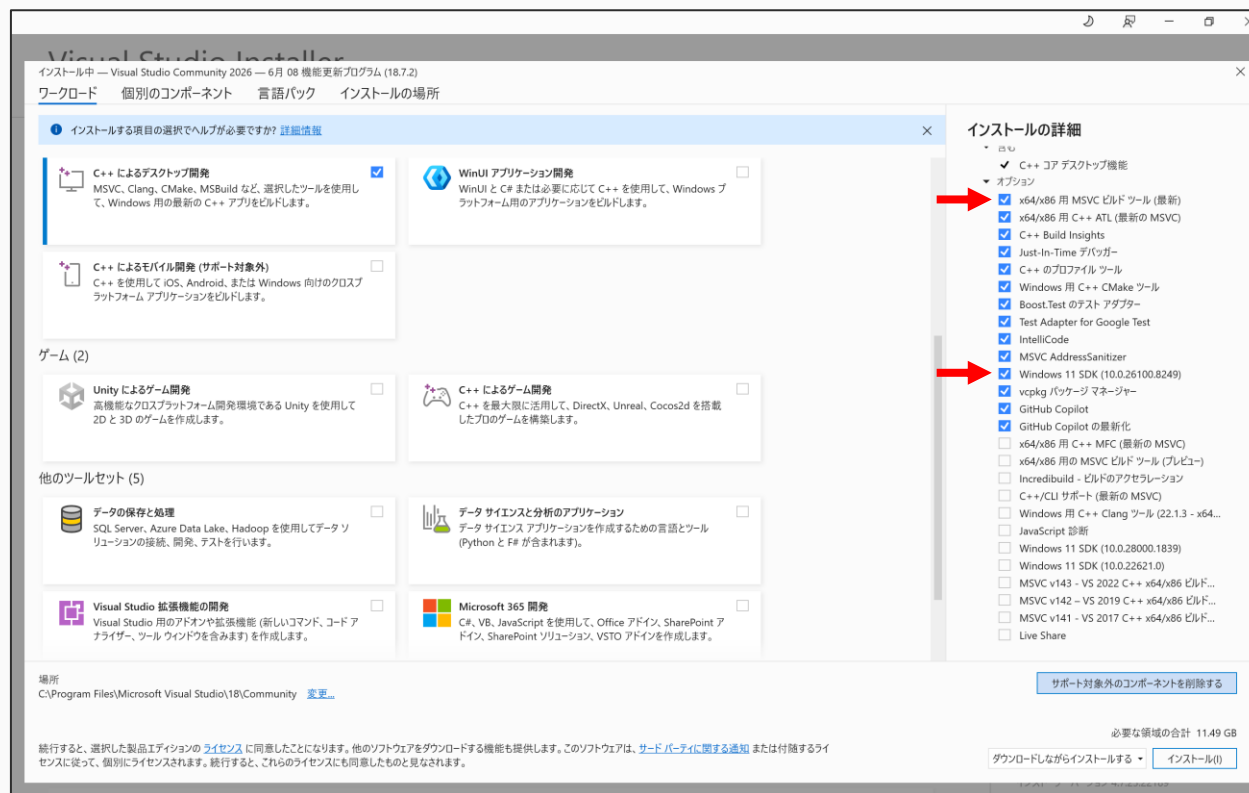
※起動したけどなんか変 / 起動しない→FAQを参照

Tips : Tabを活用しよう

これらの入力を行うのは手間ではないだろうか？
ターミナルでファイル名を入力する時、Tabキーを押せば補完・候補を表示できる。tar -xvf rooまで入力してTabを押してみよ。補完できなければファイルが存在しないか、そこまで誤字をしている。

ビルド済みバイナリ編 for Windows

- ① [Visual Studio 2022](#) インストーラをダウンロードする。
- ② インストーラを起動し、「Visual Studio Community > C++によるデスクトップ開発」を選択。
- ③ 右のチェックボックスのうち、以下をチェックし、インストール。
 - x64/x86用MSVCビルドツール（最新）
 - Windows 11 SDK

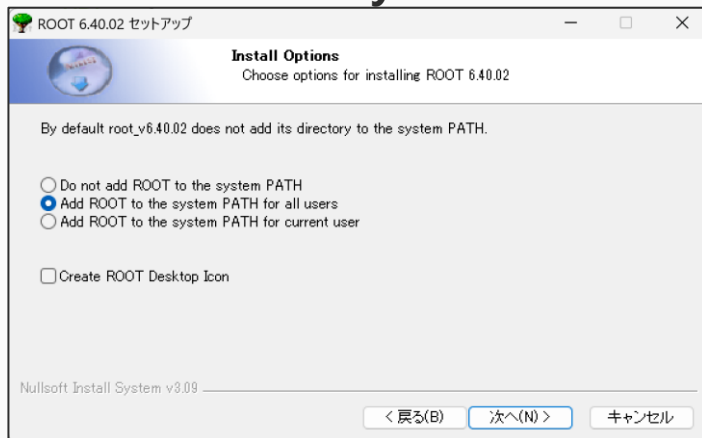


今後C++で開発をしたい人はデフォルトのままでもよい。また、後から必要に迫られたら、再インストールすればよい。

ビルド済みバイナリ編 for Windows

④ [ROOTインストーラ](#)をダウンロードし、実行。⑤⑥に注意し進める。

⑤ **Add ROOT to the system PATH for all users**を選択 ※うっかり飛ばしてしまった→FAQ「起動できない」を参照



⑥ **Full Installation**を選択 (デフォルトのまま)



Windows Visual Studio 2022 32-bit x86	root_v6.40.02.win32.python311.vc17.exe	132M
Windows Visual Studio 2022 32-bit x86	root_v6.40.02.win32.python311.vc17.zip	181M
Windows Visual Studio 2025 32-bit x86 (release with debugging information)	root_v6.40.02.win32.python314.vc18.relwithdebinfo.exe	438M
Windows Visual Studio 2025 32-bit x86 (release with debugging information)	root_v6.40.02.win32.python314.vc18.relwithdebinfo.zip	696M
Windows Visual Studio 2022 64-bit x64	root_v6.40.02.win64.python311.vc17.exe	137M
Windows Visual Studio 2022 64-bit x64	✗ root_v6.40.02.win64.python311.vc17.zip	188M
Windows Visual Studio 2025 64-bit x64 (release with debugging information)	root_v6.40.02.win64.python314.vc18.relwithdebinfo.exe	447M
Windows Visual Studio 2025 64-bit x64 (release with debugging information)	root_v6.40.02.win64.python314.vc18.relwithdebinfo.zip	709M

普通はWindows Visual Studio 2022 64-bit x64を選択。一部、古いPCを使用している場合は、CPUのアーキテクチャがx32の場合がある。不安であれば「設定>システム>バージョン情報>デバイスの仕様」で、システムの種類にx64 ベース プロセッサと書かれていることを確認する。x32であれば32-bit x86用のインストーラをダウンロードする。ARMの方（Snapdragon Xなど）→FAQを参照

⑦ rootを実行し、起動確認。

.qで終了する。早くできた人は、demoで機能を試してみるとよい。
※起動したけどなんか変 / 起動しない→FAQを参照

ビルド済みバイナリ編 for macOS

- ① CPUのアーキテクチャとOSを確認する（Intel Mac、Appleシリコンなど聞いたことがあるだろう）

「 > このMacについて」からプロセッサ（チップ）とOSを確認。

- ② [公式サイト](#)から自分の環境にあったインストーラをダウンロード。

macOS 14.8 x86_64 Xcode 16	root_v6.40.02.macos-14.8-x86_64-clang160.pkg	477M
macOS 14.8 x86_64 Xcode 16	root_v6.40.02.macos-14.8-x86_64-clang160.tar.gz	312M
macOS 15.7 arm64 Xcode 17	root_v6.40.02.macos-15.7-arm64-clang170.pkg	463M
macOS 15.7 arm64 Xcode 17	root_v6.40.02.macos-15.7-arm64-clang170.tar.gz	308M
macOS 26.5 arm64 Xcode 21	root_v6.40.02.macos-26.5-arm64-clang210.pkg	479M
macOS 26.5 arm64 Xcode 21	root_v6.40.02.macos-26.5-arm64-clang210.tar.gz	320M

- ③ 一度起動する。ブロックされるので、

「設定>プライバシーとセキュリティ>セキュリティ」から「このまま開く」

- ④ インストーラの指示に従い、インストールする。

- ⑤ 完了後、ターミナルで`root`を実行して、起動確認。

`.q`で終了する。早くできた人は`.demo`で機能を試してみるとよい。



C / C++の書き方：変数と関数の型

C言語はPythonと異なり、変数にどのようなもの（型）を代入するか、あらかじめ予告（宣言）する。

```
int i = 46;           // 「iは整数(integer)です。46を代入します」
double pi = 3.14      // 「piは倍(double)精度浮動小数点数です。3.14を代入します」
```

数学同様、関数は値（戻り値）を返す： $f(x) = x^2$ という関数は $x = 2$ を渡すと $f(2) = 4$ を返す。

```
// 2乗を計算する関数を定義し、計算結果を表示しよう
double f(double x) {           // 「小数を扱う関数f(x)を定義する。xは小数を扱う変数である」
    double y = x * x;          // 「小数を扱うyを定義し、x*xを代入する」
    return y;                  // まだ計算しただけ。「結果を返す(return)」
}

int main() {
    double i = 46.2;
    double j;
    j = f(i);                  // f(x)は計算結果をreturnするので、受け取る
    cout << "square of " << i << " is " << j << endl;      // 表示する
    return 0;                  // 右のTips参照
}
```

Tips：main関数の型

mainは実行の本体であり、実行すれば戻り値はいらないのに、なぜvoidではなくintなのだろうか。①昔はvoid型がなかったので、形式的にreturn 0を書いてint型ということにした ②0は真偽判定で偽を意味するので、実行中ですか→いいえ、の判定に使える。

もし関数が表示までしてくれるなら？

```
void g(double x) {
    double y = x * x;
    cout << "square of " << x << " is " << y << endl;
}
```

main関数で受け取る必要がなくなる。受け取らないのでreturnもない。つまり戻り値はいらない→void型

このフォルダで使うROOTマクロ

READMEにある練習を、実際の4ファイルで進める。

`example/practice_exp.dat`

指数分布の入力データ

`example/exercise.C`

C++ ROOTマクロ

`example/exercise.py`

同じ解析のPyROOT版

`example/make_practice_data.py`

入力データを作り直すPythonコード

実行

```
python example/make_practice_data.py  
root -l -q example/exercise.C+
```

まずC++版を読み、最後にPyROOT版を見る。

1. practice_exp.datを読む

exercise.Cは、自分のファイル位置を基準に入力データを探す。

入力場所

```
const std::string script_dir =  
    gSystem->DirName(__FILE__);  
const std::string input_path =  
    script_dir + "/practice_exp.dat";  
  
const auto values = read_values(input_path);
```

read_values

```
std::ifstream input(path);  
std::vector<double> values;  
double value = 0.0;  
  
while (input >> value) {  
    values.push_back(value);  
}
```

ポイント

1行ずつdoubleとして読み、vectorに追加する。スクリプト位置を基準にするので、同じフォルダ構成なら別のPCでも動かしやすい。

2. TH1Dを作ってデータを入れる

exercise.C

```
TH1D hist_expo(  
    "hist_expo",  
    "Exponential data;x;Counts",  
    100, 0.0, 100.0  
);  
  
for (const double value : values) {  
    hist_expo.Fill(value);  
}
```

TH1Dの引数

名前	hist_expo
----	-----------

軸タイトル	x / Counts
-------	------------

ビン数	100
-----	-----

横軸	0 ~ 100
----	---------

Fill

値ごとにFillを呼ぶと、対応するビンのカウントが1増える。

3. ROOTのexpoでフィットする

exercise.C

```
const TFitResultPtr result =  
hist_expo.Fit("expo", "SQ", "", 0.0, 100.0);
```

expo はROOTに用意されている指数関数

$$\exp([0] + [1] x)$$

Fitオプション

S: フィット結果を返す
少なくする

Q: ターミナルへの表示を

寿命との関係

傾き[1]が負なら、 $\tau = -1 / [1]$ で減衰定数を求められる。

4. 自分で減衰関数を定義する

exercise.C

```
TF1 decay(  
    "decay",  
    "[0]*exp(-x/[1])",  
    0.0, 100.0  
);  
  
decay.SetParameters(1000.0, 10.0);  
hist_decay.Fit(&decay, "SQ");
```

パラメータ

[0] N_0 : $x = 0$ 付近の大きさ

[1] τ : 減衰の時間スケール

初期値

SetParametersで N_0 と τ の出発点を与える。初期値が離れすぎると、うまく収束しないことがある。

5. Canvasに結果を並べる

exercise.C

```
TCanvas canvas("canvas", "ROOT exercise", 1200, 500);
    canvas.Divide(2, 1);

    canvas.cd(1);
    hist_expo.Draw();

    canvas.cd(2);
    hist_decay.Draw();
```

左 ROOTのexpo

右 自分で定義した関数

比較すること

2つのフィットから得た τ が一致するか、フィット結果とヒストグラムの形を見比べる。

画像保存の処理もexercise.Cに含まれている。

実行して結果を確認する

実行

```
python example/make_practice_data.py  
root -l -q example/exercise.C+
```

確認する値

expoの傾き 約 -0.102

-1 / 傾き 約 9.84

自作関数の τ 約 9.84

うまく動かないとき

入力データの場所、ROOTとC++コンパイラの組み合わせ、末尾の+を確認する。

次に試すこと

ビン数、横軸の範囲、フィット範囲を変えて、結果がどの程度変わるかを見る。

Practice : TDCデータを解析するにはどう書けばいいだろう？

測定側のコードはある。次は、時間差を解析する流れを自分で考える。

測定側

```
oscillo_Scripts/create_TDC.py  
oscillo_Scripts/create_TDC_new.py
```

書き始め

まずはexercise.Cの「読む → Fillする → Fitする → Drawする」をTDCの時間差へ置き換える。

解析側で考えること

- どの列を時間差として読むか
- 秒・ns・ μ sのどれで扱うか
- ヒストグラムの範囲とビン数
- フィット関数とフィット範囲
- 背景や0付近のピークをどう扱うか

ゴール

寿命らしい値が出るかだけでなく、単位やフィット範囲を変えたときの安定性も確認する。

C++のあとに：PyROOT

example/exercise.pyは、同じ解析をPythonからROOTへ渡している。

- Python対応のROOTを使う
- ROOTに対応するPythonを使う
- 仮想環境を用意しておく与管理しやすい

Windowsで実行

```
.venv-root/Scripts/python.exe example/exercise.py
```

位置づけ

C++マクロを理解したあと、必要な部分だけPyROOTで試す。

example/exercise.
py

```
import ROOT

hist = ROOT.TH1D(
    "hist_expo", "Exponential data;x;Counts",
    100, 0.0, 100.0
)
decay = ROOT.TF1(
    "decay", "[0]*exp(-x/[1])", 0.0, 100.0
)
hist.Fit(decay, "SQ")
```

- CentOSでもできるか？

⇒ できるが、すでにサポートが終了している。

- パッケージが見つからない： `Error: Unable to find a match: ...` `E: Failed to fetch http://...`

⇒ まずはパッケージマネージャが古いことを疑う・・・ `sudo dnf update` / `sudo apt update`

それでもダメなら[データベース](#)でパッケージ名を検索し、自分のOSのページからPackage nameを探す。

例えばAlmaLinux 10ではlibXft-develだが、Ubuntuではlibxft-dev。OSやバージョンによって名前が変わることがある。公式サイトやスライドの情報が古い場合があるので、このように検索されたい。

- PATHを通すとは？ `echo 'source ~/opt/root/bin/ROOTの環境設定スクリプト' >> ~/シェル設定` は何をしている？

⇒ ROOTの環境設定スクリプトはROOTを起動するためのスクリプト。これをコンピュータに認識させることで、`root`と打てばROOTが起動するようになる。これを「PATHを通す」という。

公式サイトではPATHの通し方； `source ...`しか書かれていないが、これではターミナルを開くたびにPATHを通さなければならない。`echo ...`はこれをシェル設定という、自動実行される設定ファイルに追記して、明日以降の手間を省いてる。

Windowsではこの通りにできないので、次頁の「起動できない」を参照。

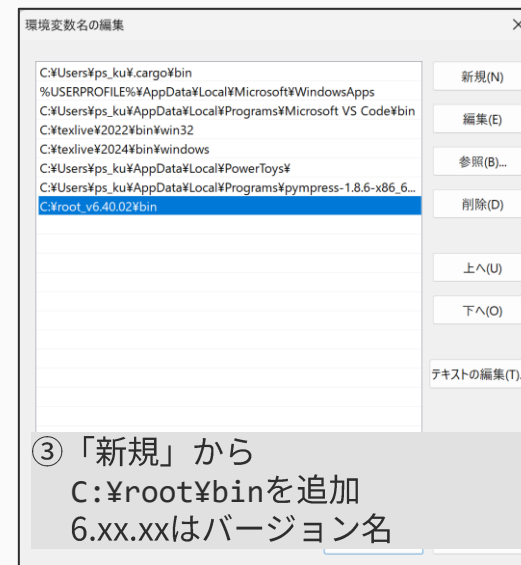
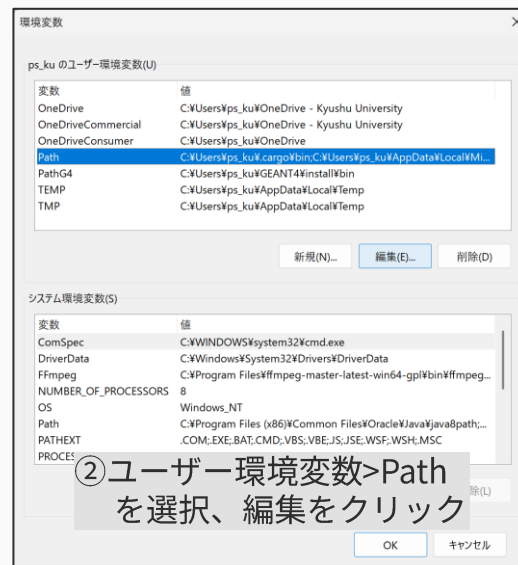
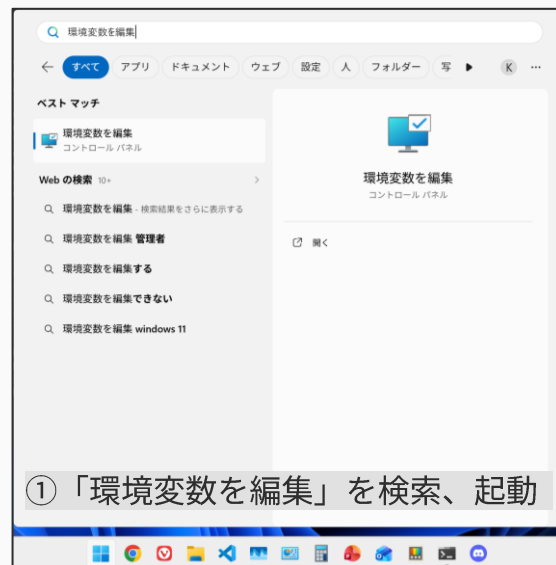
- 起動できない：`-bash: root: command not found` `root: command not found` (AlmaLinux / Ubuntu)

‘root’ は、内部コマンドまたは外部コマンド、
操作可能なプログラムまたはバッチ ファイルとして認識されていません。 (Windows)

⇒ ビルド済みバイナリの場合、PATHが通っていない。

Linux: `echo 'source ~/opt/root/bin/ROOTの環境設定スクリプト' >> ~/シェル設定`を実行。

Windows: 環境変数を編集する。



- 起動できるが右のようなエラーが出る

⇒ ROOT/ClingがC++標準ライブラリヘッダを見つけられていない。

C++のコンパイラが存在しない可能性が高い。

Linux : `sudo dnf install gcc-c++ gcc / sudo apt install g++ gcc`

Windows : Visual C++をインストールする。

Visual Studio 2022インストーラをダウンロード・起動し、

「Visual Studio Community 2022 > C++によるデスクトップ開発」

を選択。以下をインストールする。

- x64/x86用MSVCビルドツール（最新）
- Windows 11 SDK

```
PS C:\Users\ps_ku> root
RegOpenKeyEx(HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Microsoft SDKs\Windows
^
RegQueryValueEx(KitsRoot10): returned 2:
warning: unable to find a Visual Studio installation; try running Clang
input_line_1:1:10: fatal error: 'new' file not found
#include <new>
^~~~~~
input_line_3:51:10: fatal error: 'string' file not found
#include <string>
^~~~~~

-----
| Welcome to ROOT 6.40.02                                     https://root.cern |
| (c) 1995-2025, The ROOT Team; conception: R. Brun, F. Rademakers |
| Built for win64 on Jun 10 2026, 10:31:50 |
| From tags/v6-40-02@v6-40-02 |
| With MSVC 19.39.33521.0 std201703 |
| Try '.help'/'?', '.demo', '.license', '.credits', '.quit'/'q' |
-----

input_line_7:1:10: fatal error: 'iostream' file not found
#include <iostream>
^~~~~~
root [0]
```

- **CPUがARM系なのだが？**

⇒ Linux / WSL2なら問題ない。macOSかつビルド済みバイナリを使うなら、ARM版のインストーラを選択する。

Windowsの場合、ARM版のインストーラは用意されていない。ただしWindows on Armにはx86/x64アプリのエミュレーション機能があり、Windows10なら32-bit x86、Windows11なら64-bit x64が使えるかもしれない。インストールし、動作に不具合があればWSL2を利用する。（筆者の手元にARM機がないため検証不可。また、インストールされているPythonがARMネイティブであればPyROOTは使えない）

- **Ubuntuのバージョンに対応しているReleaseがない。**

⇒ Ubuntuのバージョン名はリリース日のYY.MMになっている（例：Ubuntu 23.04→23年4月リリース）
Ubuntuのリリース日以降の[ROOT Release](#)をまず探す。
なければ一つ前のバージョン用を試してみる。（例：Ubuntu 23.04→22.04用を試す）